VLM-MPC: Model Predictive Controller Augmented Vision Language Model for Autonomous Driving

Keke Long[1], Haotian Shi[2], Jiaxi Liu[1], Chaowei Xiao[3], Xiaopeng Li[1]*

[1] Department of Civil and Environmental Engineering, University of Wisconsin-Madison, Madison, WI, USA

[2] College of Transportation, Tongji University, Shanghai, China

[3] Information School, University of Wisconsin-Madison, Madison, WI, USA

Corresponding author: xli2485@wisc.edu

ABSTRACT

Motivated by the emergent reasoning capabilities of Vision Language Models (VLMs) and their potential to improve the comprehensibility of autonomous driving systems, this paper introduces a closed-loop autonomous driving controller called VLM-MPC, which combines a VLM with Model Predictive Controller (MPC) to evaluate how model-based control could enhance VLM decision-making. The proposed VLM-MPC is structured into two asynchronous components: The upper level VLM generates driving parameters (e.g., desired speed, desired headway) for lower-level control based on front camera images, ego vehicle state, traffic environment conditions, and reference memory; The lower-level MPC controls the vehicle in real-time using these parameters, considering engine lag and providing state feedback to the entire system. Experiments based on the nuScenes dataset and Carla simulation validated the effectiveness of the proposed VLM-MPC across various environments (e.g., night, rain, fog, and intersections). The results demonstrate that the VLM-MPC consistently maintains Post Encroachment Time (PET) above safe thresholds, in contrast to some scenarios where the VLM-based control posed collision risks. Additionally, the VLM-MPC enhances smoothness compared to the real-world trajectories and VLM-based control. By comparing behaviors under different environmental settings, we highlight the VLM-MPC's capability to understand the environment and make reasoned inferences. Moreover, we validate the contributions of two key components, the reference memory and the environment encoder, to the stability of responses through ablation tests.

Keywords: Autonomous Vehicle, Foundation Model, Large Language Model, Vehicle Safety

1 INTRODUCTION

The comprehensibility of autonomous driving systems is of paramount importance. Existing autonomous driving systems, whether model-based (Gao et al., 2022; Shang et al., 2022) or learning-based (Shi et al., 2021), typically require complex rules or reward function designs to understand the traffic environment and driving preferences. However, this reliance on predefined rule sets often limits their adaptability to various traffic scenarios (Tian et al., 2024). Furthermore, learning-based autonomous driving systems face the challenge of out-of-distribution (OOD) data (Filos et al., 2020), where limited datasets fail to ensure reliable decision-making in rare real-world driving situations.

Given these challenges, language models (LMs), such as Large Language Models (LLMs) and Vision-Language Models (VLMs), show potential as a promising solution (Valmeekam et al., 2023; Wang et al., 2025). Leveraging extensive datasets of text and images, LMs have endowed them with reasoning capabilities (Brown et al., 2020; Yingzi Ma et al., 2023), enhancing the interpretability and performance of autonomous driving systems in complex environments. Recent studies have begun utilizing VLMs and LLMs to process driving environmental inputs, such as images and the positions of traffic participants. This information is encoded as language representations and input into LMs, enabling them to perform driving tasks effectively.

The application of LMs in autonomous driving tasks encompasses various roles. Depending on the control mode output by the LM, current LM-based vehicle controllers can be classified into two categories, as outlined in TABLE 1. The first category of approach, "LM to action" (Sha et al., 2023), directly uses LMs to generate autonomous vehicle (AV) actions, such as waypoints, throttle, and pedal commands. This is an 'end-to-end' autonomous driving mode. However, these studies often fail to account for vehicle kinematics and dynamics in LM models (Mao et al., 2023), raising concerns about the real-world applicability of the generated actions. Additionally, this approach poses challenges to the response speed of LMs. In autonomous driving, particularly in near-collision scenarios, a control frequency of approximately 10Hz is required (Campolo et al., 2017). However, current LMs are incapable of achieving such high response speeds, further limiting their practicality in real-world applications. Some studies have sacrificed model accuracy by selecting smaller models for faster response times.

To leverage LMs' reasoning capabilities while mitigating the issue of slow response speeds, some AV control studies have adopted the second category of approach "LM to commands/codes". In this approach, LMs serve as upper-level decision-makers, providing instructions or commands to the lower control system. This method decouples upper-level LM from lower-level vehicle control, allowing the upper-level LM to focus on long-term planning at a slower frequency. Meanwhile, the lower level inherits mature control schemes to perform high-frequency control. This asynchronous control approach offers two key benefits. First, it reduces the necessity for high response speeds from the LM, making it more compatible with the current capabilities of LMs (Chen et al., 2024). Second, it allows the lower-level control to leverage well-established vehicle control schemes, such as model predictive control (MPC)-based trajectory tracking (Long et al., 2024; Wang et al., 2021), which are widely adopted in the industry.

However, a fundamental question remains unanswered: does incorporating a model-based approach like MPC into an LM benefit the LM's performance in autonomous driving? To tackle this unsolved question, we propose an autonomous driving controller with a two-level structure: VLMs, representing LM, serve as the upper-level vehicle control, while MPC functions as the lower-level controller. The upper-level VLM processes inputs such as the vehicle's front camera images, textual environment descriptions, and reference memory to generate control parameters for the lower-level MPC. The MPC then uses these parameters, accounting for vehicle dynamics with engine lag, to achieve realistic vehicle behavior and provide state feedback to the VLM. This asynchronous two-level structure effectively mitigates the challenge of slow VLM response speeds. We compare this controller with the VLM-to-action approach to evaluate the impact of integrating MPC within a VLM framework.

TABLE 1 Related Research

	Research	Language model input	Foundation model output	Open/ Closed loop	Data/ Platform
LM to action	OmniDrive (S. Wang et al., 2024)	3d vision/traffic info/trajectory memory/instruction	Planned waypoint	Open	nuScenes
	3D-Tokenized LLM (Bai et al., 2024)	3d vision/traffic info/trajectory memory/instruction	Planned waypoint	Open	nuScenes
	GPT-Driver (Mao et al., 2023)	Text scene description (status of ego vehicle and surrounding vehicle)	Description of the action; trajectory	Open	nuScenes
	Language Agent (Mao et al., 2024)	Text scene description (status of ego vehicle and surrounding vehicle)	Description of the action; trajectory	Open	nuScenes
	DriveGPT (Xu et al., 2024)	Videos, text instructions	Control (speed, turning angle)	Open	CC3M; WebVid-2M; BDD-X
	BEVGPT (P. Wang et al., 2024)	Bird's-eye-view (BEV) images	Trajectory	Open	Lyft
	Driving with LLMs (Chen et al., 2023)	Text scene description (route, vehicle, pedestrian, ego state)	Steering/pedal	Open	Self-made open dataset
	SurrealDriver (Jin et al., 2023)	Text memory, safety criteria, text scenario description	Action (speed up/stop)	Close	Carla
	ADriver-I (Jia et al., 2023)	Historical frames, text scenario description	speed, Steer Angle	Close	nuScenes
	LMDrive (Shao et al., 2023)	Traffic info, vision, instruction	Waypoints	Close	Carla
	EMMA (Hwang et al., 2024)	Camera videos, intent command, historical ego status	Waypoints trajectory	Open	Waymo Open Motion Dataset, nuScenes
	DriveVLM (Tian et al., 2024)	Traffic info/vision/ instruction	Waypoints; reference trajectory for tracking	Close	nuScenes, SUP-AD dataset
LM to comm-ands/codes	DiLu (Wen et al., 2023)	Text Text scene description	Five operational actions	Open	Highway-env, CitySim
	Personalized autonomous driving (Cui et al., 2024b)	Text scene description, instructions, API docs	Language model program	Open	Self-made open dataset
	languageMPC (Sha et al., 2023)	Text scene description	MPC parameters	Open	IdSim
	LLM-Assist (Sharan et al., 2023)	Driving parameters	Trajectory planner parameter	Close	nuPlan

PlanAgent (Zheng et al., 2024)	BEV map, text scene description	Planning parameters	Close	nuPlan
Co-driver (Guo et al., 2024)	Front camera image, text scene description	Control type, control parameter	Close	Carla
Asynchronous llm (Chen et al., 2024)	Feature for trajectory planner	Driving parameters	Close	nuPlan
DriVLMe (Huang et al., 2024)	Perception video, text history action, planned route, human instruction	Eight operational actions	Close	Carla
Drive as You Say (Cui et al., 2024a)	Text scene description	Five operational actions	Close	HighwayEnv
LaMPilot (Yunsheng Ma et al., 2023)	Text scene description, instructions, API docs	Code	Close	Self-made open dataset
LLaDA (Li et al., 2024)	Execution plan, traffic code, scene description	Motion plan in natural language	Open	nuScenes, nuPlan

Another critical problem for LMs is the occurrence of hallucinations in their outputs (Farquhar et al., 2024). Hallucinations are often defined as LLMs generating 'nonsensical or unfaithful content to the provided source content' (Ji et al., 2023). These can arise for various reasons, including biases in the training data, lack of context awareness, and inherent limitations in the model's architecture. These hallucinations hinder the application of LMs across various fields, including autonomous driving, where they may fail to accurately comprehend the driving environment, leading to inconsistent responses or the generation of misleading information. Currently, a systematic evaluation to verify solutions that could enhance the reliability of LM responses in autonomous driving tasks is relatively rare.

In summary, current studies in LM research for autonomous driving control have not discussed the contribution of model-based controllers like MPC to LM-based control and lack a discussion about the methods for improving the reliability of LM responses in autonomous driving. To fill this gap, we propose a VLM-MPC framework for autonomous driving. The proposed VLM-MPC is structured into two asynchronous levels: The upper level VLM generates driving parameters for lower-level control, and The lower-level MPC controls the vehicle in real-time using these parameters.

To evaluate the performance of VLM-MPC against the LM-to-action approach, a real-world driving scenarios dataset and traffic simulator are adopted for testing. The results demonstrate that VLM-MPC ensures safety and improves smoothness across all tested scenarios, whereas the baseline LM-to-action method poses safety risks. To further validate the proposed VLM-MPC, we extend the analysis to a different variation: the LLM-MPC framework, which also demonstrates safety and improved smoothness across tested scenarios. Furthermore, we emphasize environmental understanding by VLM by examining its performance in different scenarios and the visualization of VLM attention to the image input. Results show that the VLM exhibited appropriate variations in behavior under different environmental conditions.

To avoid hallucination and improve the response stability of VLM, the VLM-MPC framework incorporates two key components: the reference memory and the environment encoder. The reference memory compiles average parameters applicable to current scenes based on their characteristics. The environment encoder extracts vital environmental factors such as weather, lighting, and road conditions from images captured by the vehicle's front camera. These two components provide additional context to the VLM, thus mitigating the risk of incorrect or nonsensical responses. Ablation tests validate that both components significantly enhance the response stability of VLM, thereby improving the overall performance and reliability of VLM-MPC.

The structure of this paper is as follows. Section 2 describes the proposed VLM-MPC method. Section 3 shows the experiment settings and results. Section 4 concludes the paper and discusses the limitations of the proposed method and future research. Our code and data will be made available upon acceptance of this paper.

2 METHODOLOGY

This section presents VLM-MPC, a VLM-based controller for autonomous driving. We first introduce the overall architecture of VLM-MPC in Section 2.1. Then, we introduce the two levels of our method: upper level VLM (Section 2.2) and lower level MPC controller (Section 2.3)

2.1 Overall Architecture

The overall pipeline of the proposed closed-loop VLM-MPC is shown in Figure 1. The VLM-MPC is structurally divided into two levels: an upper-level VLM part serves as an upper-level planner, and a lower-level MPC part for autonomous vehicle control.

Upper level VLM: The upper level relies on VLM as the core reasoning and decision-making component, consisting of four subcomponents: 1) Environment encoder (Section 2.2.1): Generates environment descriptions from the vehicle's front camera. 2) Scene Encoder (Section 2.2.2): Extracts information about the ego vehicle and preceding vehicles from the dataset. 3) Reference memory (Section 2.2.3): This component stores aggregated driving parameters from the history driving dataset, providing the VLM with historical context and average parameters for varied driving scenarios. 4) Prompt Generator: Constructs prompts for input and guides the VLM to perform chain-of-thought (CoT) analysis. The first three components collect information to construct the prompt for the fourth component. Once the prompt is input into the VLM, it produces the upper-level results, specifically the key driving parameters.

Lower level MPC: The lower level is a rule-based MPC. This part considers the vehicle's underlying dynamics. It relies on the key driving parameters set by the upper level to determine specific actions, which are then transmitted to the vehicle for execution (Section 2.3.1).

The upper and lower levels of the VLM-MPC are decoupled and operate at different frequencies. This setup enables controlled asynchronous inference. Thus, the upper-level VLM is not required to process at a high frequency as the lower-level MPC. During asynchronous intervals, the previously derived upper-level instruction and driving parameters continue to guide the lower-level MPC.

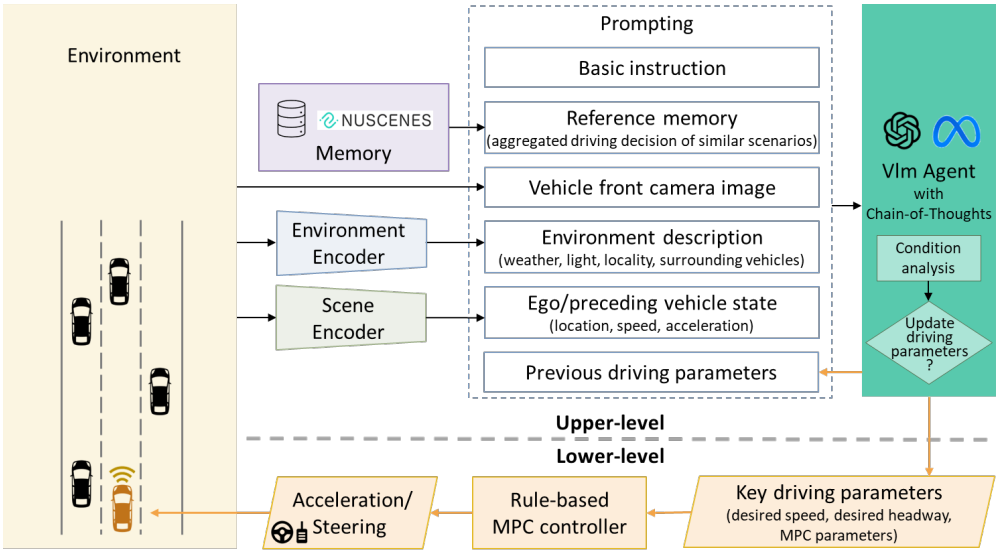


Figure 1 Architecture of VLM-MPC.

2.2 Upper level: VLM

This section introduces how the upper level: VLM, collects scene information and constructs prompts for the VLM to generate key driving parameters. This section introduces each component in sequence: reference memory dataset, environment encoder, scene encoder, and prompt generator.

2.2.1 Environment Encoder

Driving environment characteristics, such as weather, light, and road conditions, significantly influence driving performance. Let $\mathcal{T} \coloneqq \{t_1, t_2, \ldots, t_n\}$, where n represents the total number of time points required to complete a driving task within one scene. To enhance the environmental understanding of VLM, this model outputs a linguistic description E_t of the driving environment at time $t \in \mathcal{T}$. E_t includes 6 detailed scene descriptions at time t:

$$\mathrm{E}_t \coloneqq \left\{\mathrm{E}_t^{\mathrm{Wea}}, \mathrm{E}_t^{\mathrm{Lig}}, \mathrm{E}_t^{\mathrm{RT}}, \mathrm{E}_t^{\mathrm{RC}}, \mathrm{E}_t^{\mathrm{Obs}}\right\}, t \in \mathcal{T} \tag{1}$$

The Contrastive Language-Image Pre-training (CLIP) model (Fang et al., 2024; Radford et al., 2021) is adopted to extract E_t from the vehicle front camera imagesA pre-trained 'ViT-B/32' configuration is loaded. Several categories of descriptive texts were defined, including weather conditions, lighting, road types, road conditions, and static obstacles. The process begins by preprocessing and encoding an input image using the CLIP model to obtain image features. These features are then compared with pre-defined descriptive texts through cosine similarity calculations between the image and text features encoded by the CLIP model, as shown in Figure 2.

In this setup, descriptions of weather conditions $\mathrm{E}_t^{\mathrm{Wea}}$, lighting $\mathrm{E}_t^{\mathrm{Lig}}$, and road types $\mathrm{E}_t^{\mathrm{RT}}$ are outputted regardless of the confidence score, ensuring these essential scene attributes are consistently provided. For road conditions $\mathrm{E}_t^{\mathrm{RC}}$ and obstacles $\mathrm{E}_t^{\mathrm{Obs}}$, the descriptions are only outputted if their confidence scores exceed specific thresholds (0.3 and 0.2, respectively). This conditional output prevents the inclusion of less specific information, maintaining the accuracy and relevance of the extracted features.

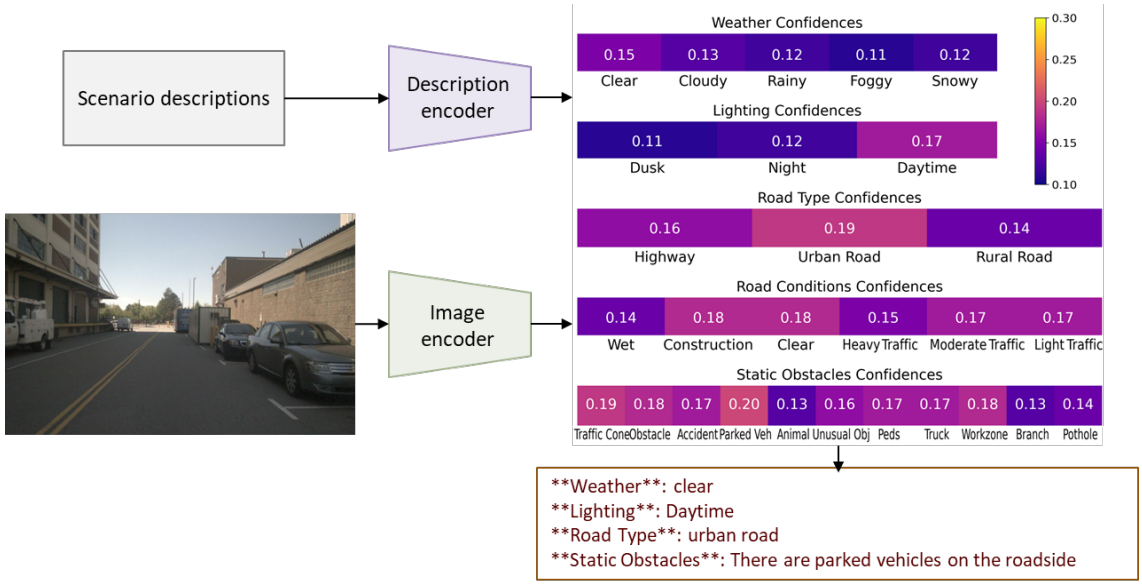


Figure 2 CLIP model structure in environment encoder

2.2.2 *Scene Encoder*

To enhance the VLM's understanding of the driving task, the prompt includes the historical state information of the ego vehicle and the states of significant surrounding vehicles and other obstacles. Significant surrounding vehicles refer to the preceding vehicle that the MPC algorithm needs to consider for safe and efficient car-following behavior. Other obstacles include any objects that obstruct the ego vehicle's path, such as pedestrians, and elements that need to be considered due to traffic rules, such as stop lines at red lights or stop signs at intersections.

The scene encoder extracts the scene status S_t at time t, including the ego vehicle state s_t^{ego}, the preceding vehicle state s_t^{pre}(if applicable), and the position of the stop line s_t^{SL} (if applicable):

$$\mathrm{S}_t \coloneqq \{s_t^{\mathrm{ego}}, s_t^{\mathrm{pre}}, s_t^{\mathrm{SL}}\}, t \in \mathcal{T} \tag{2}$$

$$s_t^{\mathrm{ego}} \coloneqq \left[x_{t' \in \mathcal{T}_t}^{\mathrm{ego}}, v_{t' \in \mathcal{T}_t}^{\mathrm{ego}}, a_{t' \in \mathcal{T}_t}^{\mathrm{ego}}\right]_{\mathcal{T}_t = \{t-\gamma\Delta t, \cdots, t-\Delta t, t\}}, t' \in \mathcal{T} \tag{3}$$

$$s_t^{\mathrm{pre}} \coloneqq \left[\Delta x_t^{\mathrm{pre}}, v_t^{\mathrm{pre}}, a_t^{\mathrm{pre}}\right], t \in \mathcal{T} \tag{4}$$

$$s_t^{\mathrm{SL}} \coloneqq \left[\Delta x_t^{\mathrm{SL}}\right], t \in \mathcal{T} \tag{5}$$

where s_t^{ego} are the γ steps of historical states of the ego vehicle containing vehicle position, vehicle speed, and vehicle acceleration at the previous time set $\mathcal{T}_t = \{t - \gamma\Delta t, \cdots, t - \Delta t, t\}$, Δt is the historical state sample interval. In this research, $\gamma = 5$, $\Delta t = 0.5s$. s_t^{pre} is the status information of the preceding vehicle at time t. Δx_t^{SL} is the vehicle position in Frenet coordinates (Reiter and Diehl, 2021), with the ego vehicle as the origin. s_t^{SL} is the status information of the stop line, s_t^{SL} exists when the lane of the ego vehicle is blocked by a traffic signal or stop sign.

The preceding vehicle's information is determined based on vehicle positions, road information, and other vehicle positions from the nuScenes dataset (Caesar et al., 2020). Similarly, stop line positions are derived from the map information provided in the nuScenes dataset. This study does not account for errors in the perception phase of traffic participants.

2.2.3 *Reference Memory*

Reference memory is constructed based on real vehicle trajectories from the nuScenes dataset. It serves as a reference for generating the key driving parameters by aggregating data from various driving scenes. Although the trajectories from nuScenes are not exclusively from autonomous driving or human driving, they generally exhibit safe driving behaviors, as supported by the safety analysis in the experiment section. Thus, we consider these trajectories valuable references for determining key driving parameters.

The key driving parameters include six parameters: $[N, Q, R, Q^h, v^d, h^d]$:

1. **Prediction horizon** N: The number of time steps the controller looks ahead.
2. **Speed maintenance weight** Q: Weight for maintaining the desired speed. Considering there are three weights in the system, Q is fixed at 1.
3. **Control effort weight** R: Weight for minimizing control effort.
4. **Headway maintenance weight** Q^h: Weight for maintaining the desired headway (distance) to the front vehicle.
5. **Desired speed** v^d: The target speed (m/s) for the ego vehicle.
6. **Desired headway** h^d: The desired headway (s) between the ego vehicle and the front vehicle.

The six parameters are selected to optimize autonomous vehicle control. The prediction horizon N anticipates future changes, enabling proactive adjustments. Speed maintenance weight Q and control effort weight R balance desired speed with minimal control effort for efficient driving. Headway maintenance weight Q^h ensures a safe distance from the vehicle ahead, which is crucial for collision avoidance and

traffic flow. Desired speed v^d and desired headway h^d set specific performance targets. Together, these parameters harmonize safety, efficiency, and comfort, enhancing the driving experience.

To obtain the initial parameter values for each scene, we extracted driving behavior statistics from the ego vehicle human-driven trajectories provided in the nuScenes dataset. For each scene, we calculate the target speed, headway, and relevant MPC weights by analyzing how the ego vehicle accelerates, maintains distance, and responds to the lead vehicle. These calibrated values serve as the basis for constructing reference memory entries.

The reference memory dataset encapsulates these parameters using the notation $M(\mathrm{E_t})$, the function M maps each scenario to a tuple of averaged MPC parameters specific to that scenario:

$$M(\mathrm{E_t}) = \left[\bar{N}(\mathrm{E_t}), \bar{Q}(\mathrm{E_t}), \bar{R}(\mathrm{E_t}), \bar{Q}^h(\mathrm{E_t}), \bar{v}^d(\mathrm{E_t}), \bar{h}^d(\mathrm{E_t})\right], t \in \mathcal{T} \tag{6}$$

Here $\bar{N}(\mathrm{E_t})$ denote the average values of the prediction horizon for all scenarios that share the same environment characteristics of $\mathrm{E_t}$, Similarly, $\bar{Q}(\mathrm{E_t}), \bar{R}(\mathrm{E_t}), \bar{Q}^h(\mathrm{E_t}), \bar{v}^d(\mathrm{E_t})$ and $\bar{h}^d(\mathrm{E_t})$ represent the average values of the speed maintenance weight, control effort weight, headway maintenance weight, desired speed, and desired headway, respectively, for those scenarios.

To enable efficient scene similarity search, we adopt a fast and scalable k-nearest neighbor retrieval algorithm based on CLIP image embeddings and FAISS. Each image is first encoded into a 512-dimensional vector using the pre-trained CLIP ViT-B/32 model. For speed consideration, all images are resized to 128×128 before encoding. The embeddings are normalized and indexed using FAISS (Johnson et al., 2021) with cosine similarity (inner product on normalized vectors).

Given a query image embedding $\boldsymbol{q} \in \mathbb{R}^{512}$ and a memory database $\mathcal{M} = \{m_1, m_2, \cdots m_N\}$, we computed its CLIP embedding and retrieve the top-k most similar images from the indexed dataset.

$$\text{top} - k(\boldsymbol{q}) = \arg\max_{m_i \in \mathcal{M}} cos(\boldsymbol{q}, m_i) = \arg\max_{m_i \in \mathcal{M}} \frac{\boldsymbol{q} \cdot m_i}{\|\boldsymbol{q}\| \cdot \|m_i\|} \tag{7}$$

where cosine similarity is used to measure visual similarity in the embedding space. This allows us to identify historical scenes that are visually close to the current view, which can serve as memory references or retrieval examples. To ensure high-speed performance, all image features are precomputed and stored, avoiding redundant computation at runtime. The average retrieval time is under 0.1s with a thousand reference images. In the implementation on nuScenes data, due to the relatively small scale of the experimental dataset, we use the top-3 nearest scenes and average their corresponding parameters as reference. However, in real-world deployment with larger databases, this number can be increased to enhance robustness, and the similarity threshold can be relaxed to further accelerate computation without significantly affecting retrieval quality.

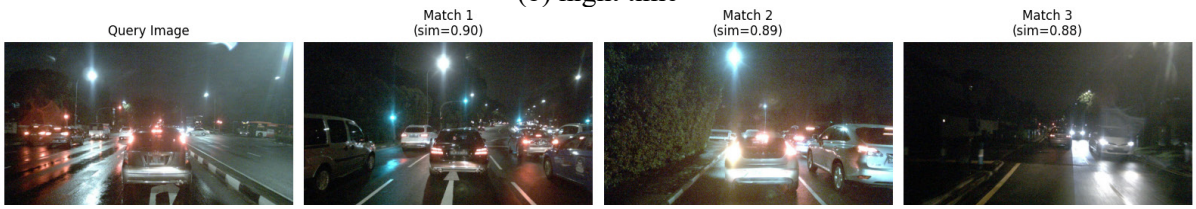
Figure 3 demonstrates visual results of our scene retrieval system under three different environmental conditions: daytime, night, and rainy night. For each query image (leftmost), we display the top 3 most similar images retrieved from the historical image database using FAISS. The cosine similarity score is shown below each retrieved result. As observed, the retrieved images preserve key scene attributes such as lighting, layout, and traffic composition, validating the effectiveness of using CLIP-based retrieval for reference memory lookup.



(a) day time



(b) night time



(c) rainy night

Figure 3 Scene matching results using FAISS-based CLIP embedding retrieval in nuScenes.

2.2.4 Prompt Generator

The prompt generator collects all relevant information from the various modules and generates the input prompts for the VLM. The role of the VLM here is not to replace perception, but to connect perception with higher-level reasoning. By combining image-based descriptions with historical memory, the VLM generates interpretable driving parameters instead of raw perception outputs. VLM generates the six necessary parameters at time t:

$$\theta_t = [N, Q, R, Q^h, v^d, h^d] \tag{8}$$

To generate these parameters, there is a slight difference in how the prompt is constructed during the initial call to the VLM for each scene and subsequent updates. For each scene, the first call at time $t = 0$ to the VLM for upper-level decision-making includes the following inputs: memory M, the front camera frame $img_{t=0}$, environment description $\mathrm{E}_{t=0}$, and the state of the ego vehicle and preceding vehicles $s_{t=0}$. The initial driving parameters $\theta_{t=0}$ are calculated as:

$$\theta_{t=0} = f^{\mathrm{VLM}}(M(\mathrm{E}_{t=0}), img_{t=0}, \mathrm{E}_{t=0}, \mathrm{S}_{t=0}) \tag{9}$$

Subsequent calls to the VLM at intervals of Δt^{u} for updating decisions, including the front camera frame img_t, environment description E_t, the state of the ego vehicle and preceding vehicles s_t, and the previous driving parameters $\theta_{t-\Delta t^{\mathrm{u}}}$. The updated driving parameters θ_t are calculated as:

$$\theta_t = f^{\mathrm{VLM}}(img_t, \mathrm{E}_t, \mathrm{S}_t, \theta_{t-\Delta t}), t \in \{\Delta t^{\mathrm{u}}, 2\Delta t^{\mathrm{u}}, \cdots\} \tag{10}$$

At the end of the prompt, we utilize chain-of-thought (CoT) prompting—presenting a series of logical and interconnected steps or directives—to help the VLM better navigate the complexities of real-world driving tasks and conduct human-like reasoning. The CoT guidance is shown in Figure 4. This method enhances the VLM's ability to handle complex scenarios by breaking the decision-making process into manageable, logical steps, improving overall performance in dynamic environments. To ensure consistency and real-time performance, the CoT guidance is pre-defined.

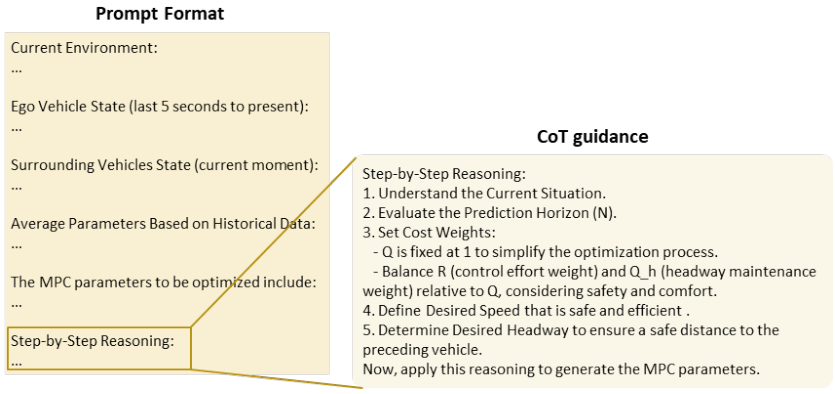


Figure 4 General prompt structure and the CoT guidance in prompts.

After generating the high-level driving parameters using the above structured prompts, we apply a rule-based validation step before passing them to the lower level MPC controller. This filter ensures that only well-structured and semantically valid outputs are used. Specifically, the upper level VLM must return a complete parameter set following a predefined format. If the output is missing value, includes only parameter ranges, or deviates from the expected structure, it is discarded. In such cases, the system continues using either the previous valid parameter set or the fallback values stored in the reference memory module. This design guarantees that the MPC always operates on safe and interpretable inputs, even when the VLM occasionally produces uncertain or incomplete outputs.

2.3 Lower level: MPC

In the lower level, we account for the vehicle dynamics, including engine lag. Building on the key parameters θ_t generated by the VLM, the MPC component optimizes the trajectory of the controlled vehicle in real time. The MPC model's primary objective is to improve traffic efficiency and safety by adjusting the vehicle's speed and position in a predictive manner. Additionally, it considers the dynamic constraints of the traffic environment and the vehicle's operational limitations, aiming to minimize the impact of traffic oscillations on the ego vehicle and, by extension, the surrounding traffic flow. The MPC continuously updates using the latest vehicle state, and together with the upper level VLM, this method helps achieve closed-loop state feedback.

2.3.1 CAV Dynamic Model

This study focuses on the longitudinal kinematics of vehicles along the driving direction. The key parameters $\theta_t, t \in \{\Delta t^{\mathrm{u}}, 2\Delta t^{\mathrm{u}}, \cdots\}$ obtained from the upper level VLM are updated with a time interval of Δt^{u}. The lower level autonomous driving requires controlling the vehicle at a finer time interval Δt^{l}, which is usually around 0.1 second, $\Delta t^{\mathrm{l}} \ll \Delta t^{\mathrm{u}}$. The lower level MPC predicts the system's behavior over a finite horizon N and plans the action over this horizon. The output of the MPC at time t is$\{u_t\}_{t\in\{t_0,t_0+\Delta t^{\mathrm{l}},\cdots,t_0+N\cdot\Delta t^{\mathrm{l}}\}}$. The detailed MPC algorithm is as follows.

The nonlinear longitudinal dynamic model of a single vehicle is described as:

$$\begin{cases} \dfrac{dx_t}{dt} = v_t \\ \dfrac{dv_t}{dt} = a_t \\ \dfrac{da_t}{dt} = f(v_t, a_t) + g(v_t)\eta \end{cases}, t \in \mathcal{T} \tag{11}$$

where x_n, v_n, a_n are position, speed, and acceleration of the vehicle n respectively, and η is the engine input. Functions f and g are given by

$$f(v_t, a_t) = -\frac{2K_d}{m} v_t a_t - \frac{1}{\tau^A}\left[a_t + \frac{K_d}{m} v_t^2 + \frac{d_m}{m}\right] \tag{12}$$

$$g(v_t) = \frac{1}{m\tau^A} \tag{13}$$

where K_d represents the aerodynamic drag coefficient, m the vehicle mass, τ^A is the engine time lag, and d_m is the mechanical drag. In this paper, we focus on the longitudinal kinematics of vehicles. Assuming the parameters in (4) (5) are priori known, we adopt the following control law structure to implement feedback linearization:

$$\eta = mu_t + K_d v_t^2 + d_m + 2\tau^A K_d v_t a_t \tag{14}$$

where u_t is the desired acceleration, determined by the upper controller. Thus, the differential equation of acceleration can be rewritten as:

$$\dot{a}_t = \frac{u_t - a_t}{\tau^A} \tag{15}$$

The objectives of CAV planning are following its preceding vehicle with a desired spacing distance and ensuring safety. Therefore, a constant time headway (CTH) spacing strategy was applied. The desired spacing distance of vehicle n is $d_t^{\mathrm{d}} = h^{\mathrm{d}} v_t + d_0$, h^{d} and d_0 are the desired constant headway and space at a standstill. Based on the CTH rule, the position error Δx_t with respect to a desired distance from the preceding vehicle $\Delta x_t = x_t^{-1} - x_t - d_t^{\mathrm{d}}$, where x_t^{-1} is the position of the preceding vehicle/stop line, and l^{-1} is the length of the preceding vehicle.

$$x_t^{-1} = \begin{cases} \min\left(x_t^{\mathrm{pre}} - l^{-1}, x_t^{\mathrm{SL}}\right), if\ both\ x_t^{\mathrm{pre}}, x_t^{\mathrm{SL}}\ exit \\ x_t^{\mathrm{pre}} - l^{-1}, \quad if\ only\ x_t^{\mathrm{pre}} exits \\ x_t^{\mathrm{SL}}, \quad if\ only\ x_t^{\mathrm{SL}} exits \end{cases} \tag{16}$$

If the information about a preceding vehicle or stop line is available, the cost function also includes a term to maintain a safe headway distance:

$$\min_{u[t_0, t_0+N]} J = \sum_{k=0}^{N-1} \left\| \Delta v_{t_0+k\cdot\Delta t^{\mathrm{l}}} \right\|_Q^2 + \sum_{k=0}^{N-1} \left\| u_{t_0+k\cdot\Delta t^{\mathrm{l}}} \right\|_R^2 + \sum_{k=0}^{N-1} \left\| \Delta x_{t_0+k\cdot\Delta t^{\mathrm{l}}} \right\|_{Q^h}^2 + \varepsilon^T \rho \varepsilon \tag{17}$$

where N the predictive horizon length. Q, R and Q^h are the weight matrix of error and input, respectively. ε is the slack vector, and ρ its weight.

When the preceding vehicle or stop line is unavailable, the objective of the MPC is to minimize a cost function J that accounts for the desired speed and control effort over a prediction horizon N. The speed error is the gap between vehicle speed and the desired speed $\Delta v_t \coloneqq v_t - v^d$. The cost function is defined as:

$$\min_{u[t_0, t_0+N]} J = \sum_{k=0}^{N-1} \left\| \Delta v_{t_0+k\cdot\Delta t^{\mathrm{l}}} \right\|_Q^2 + \sum_{k=0}^{N-1} \left\| u_{t_0+k\cdot\Delta t^{\mathrm{l}}} \right\|_R^2 + \varepsilon^T \rho \varepsilon \tag{18}$$

The optimization objective is subjected to the following constraints:

$$v_{\min} - \varepsilon \sigma_{\min}^{y} \le v_t \le v_{\max} + \varepsilon \sigma_{\max}^{y} \tag{19}$$

$$u_{\min} - \varepsilon \sigma_{\min}^{u} \le u_t \le u_{\max} + \varepsilon \sigma_{\max}^{u} \tag{20}$$

3 EXPERIMENT

To validate the proposed methodology, we conduct a series of experiments using both real-world and simulated data. Specifically, we employ the nuScenes dataset to evaluate performance in diverse but static real-world scenarios, while the CARLA simulator is used to assess the method's robustness in interactive, dynamic, and adverse conditions.

3.1 nuScenes-based Evaluation: Real-world Data

First, a real-world driving environment dataset is adopted to evaluate the proposed method and baseline methods. We utilize Llava 1.6 (Liu et al., 2024) as the foundation VLM for various components in our system. Additionally, in the sensitivity analysis, we tested the performance of GPT-4-o and GPT-4-mini. Llava 1.6 model was run on a Linux (Ubuntu) system equipped with 32GB of DDR5 RAM and an NVIDIA GeForce RTX 4090 GPU (24GB GDDR6X). The model was configured with a temperature of 0, while all other parameters were left at their default values. The GPT series models were utilized via API calls.

In the experiment, the MPC runs at a high frequency (10 Hz) for continuous low-level control, while the VLM is invoked at a lower frequency (0.2 Hz) to update the control targets. To ensure stable performance, we constrain the VLM decoding parameters to maintain a typical response time. During periods when no new VLM output is available, the MPC continues to use the most recent valid parameters, ensuring uninterrupted and safe control. This design balances the semantic reasoning power of VLMs with the real-time stability and safety of traditional controllers, making it feasible for practical deployment on current hardware. With future improvements in VLM inference speed, this architecture could support higher-level decision frequencies, further enhancing responsiveness.

3.1.1 Experiment setup

3.1.1.1 nuScenes Dataset

The nuScenes dataset is a large-scale, real-world autonomous driving dataset containing various locations and weather conditions (Caesar et al., 2020). This dataset includes data from Boston and the US, which follow right-hand traffic rules, and Singapore, which follow left-hand traffic rules. Our proposed algorithm is designed for right-hand traffic.

A total of 303 scenes are used in our evaluation, including urban road scenarios with a mix of daytime and nighttime conditions, as well as clear weather and rainy scenes. Among them, a significant portion occurs in dense traffic environments with intersections, pedestrian crossings, and traffic signals, making them suitable for evaluating safety and responsiveness in complex urban settings. We also include several nighttime scenes from Singapore, which provide additional diversity in lighting and road configurations, despite the left-hand traffic.

The heatmap in Figure 5 visualizes the p-values obtained from analyzing the differences in driving parameters across various scenarios from the nuScenes. The parameter Q does not have results because Q is fixed at 1. Significant differences are indicated with asterisks, suggesting that different parameters should be used when the corresponding features: rain, intersection, and night. Based on these findings, TABLE 2 lists the aggregated parameters selected for eight grouped scenarios. Each scenario is characterized by the presence or absence of rain, intersection, and night conditions. The table provides the final parameters for each of these grouped scenarios.

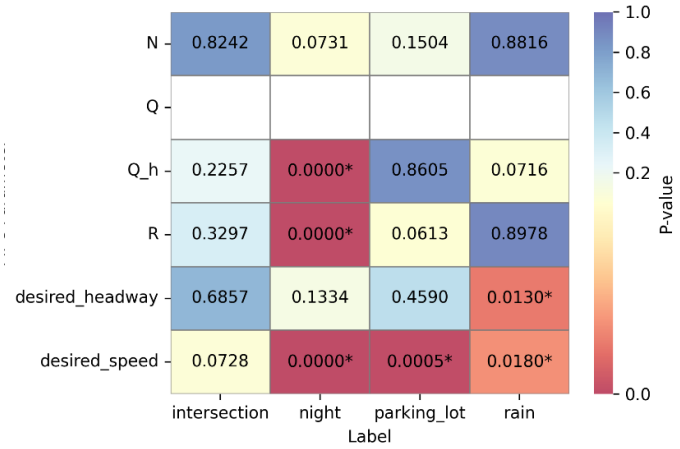


Figure 5 P-values matrix obtained from analyzing the differences in MPC parameters across various scenarios.

TABLE 2 Aggregated parameter of eight grouped scenarios.

No.	rain	intersection	night	N	Q	R	Q^h	v^d	h^d
1	0	0	0	9	1	1.68	2.75	6.44	2.60
2	0	0	1	9	1	1.68	1.99	5.09	2.55
3	0	1	0	9	1	1.68	2.75	6.44	2.60
4	0	1	1	9	1	1.15	1.99	5.09	2.55
5	1	0	0	9	1	1.15	1.99	5.09	2.55
6	1	0	1	9	1	1.68	2.75	6.44	2.60
7	1	1	0	9	1	1.68	2.75	6.44	2.60
8	1	1	1	9	1	1.68	1.99	5.09	2.55

3.1.1.2 Closed-loop Evaluation Metrics

We measure three key metrics for closed-loop evaluation from three perspectives: safety, driving comfort, and stability as follows.

1. **Safety: Post Encroachment Time (PET).** PET is a widely used surrogate safety metric that measures the time interval between the ego vehicle and another object or vehicle encroaching into the same space (Peesapati et al., 2018). A higher PET value indicates safer interactions.
2. **Smoothness: Root Mean Square of Acceleration (RMS^a).** RMS^a reflects the smoothness of the trajectory. The lower values reflect smoother, less fatiguing motion. It is calculated as:

$$\mathrm{RMS}^a = \sqrt{\frac{1}{|T|}\sum_{t=0}^{T} a_t^2} \tag{21}$$

3. **Stability: Completion rate**. This metric captures the reliability of the upper-level VLM/LLM in a closed-loop controller that operates asynchronously with a lower-level MPC. In language model-assisted control, language models may occasionally return invalid or ill-formed outputs (e.g., hallucinations, missing fields, out-of-range values). The proposed design prevents such events from immediately causing unsafe actuation: the MPC continues running with the last known valid parameters and/or reference memory until a valid update arrives. However, we still need to measure whether the upper-layer LLM generates reasonable expected parameters that can be used by the lower layer within the expected time. Operationally, we define Completion rate as the proportion of VLM/LLM invocations that pass a rule-based validator and return a complete, well-formed, and in-range parameter set by the scheduled update time (within a timeout). Because the MPC is

designed to avoid infeasible solutions under valid inputs, no MPC failures were observed; thus, this metric primarily reflects the VLM/LLM's ability to continuously produce valid parameters that enable successful scene completion.

3.1.1.3 Baseline Models

To validate the effectiveness of our proposed method, we set up three baseline models. Below are detailed explanations of each baseline model.

MPC: This baseline uses the average parameters for the given scenario derived from historical data. The parameters are fixed and do not adapt to the real-time environment. This model does not use any visual information from the vehicle's front camera. Instead, it relies on pre-calculated average parameters without leveraging dynamic memory aggregation. Although it operates in a closed-loop control manner, the control parameters remain static and do not update in real time based on feedback.

LLM to Action: This model is based on the approach proposed in (Mao et al., 2024), representing the "LM to Action" methodology in autonomous driving. The method leverages LLMs as cognitive agents to integrate human-like reasoning and decision-making into autonomous driving systems. This enables the LLM to directly generate driving actions.

LLM-MPC: This baseline integrates the LLM but does not use the advanced features of our proposed method. It serves as a comparison to assess the basic capabilities of the LLM. Unlike the proposed model, this baseline does not utilize front-camera images for decision-making. Additionally, it does not aggregate parameters from memory based on environmental characteristics, resulting in less adaptive behavior. Like the Baseline MPC Controller, it operates in a closed-loop control manner without real-time parameter updates, as the parameters are initially set and remain unchanged throughout the scenario.

The differences between the proposed VLM-MPC and the two baseline models are outlined in TABLE 3. By comparing these models, we aim to demonstrate that our proposed LLM Controller, which utilizes image inputs and reference memory, offers significant advantages in navigating complex driving scenarios and making more informed and adaptive decisions. In the future, we will conduct additional ablation studies to validate the importance of image input, memory, and reflection independently.

TABLE 3 Comparison of Proposed VLM-MPC and two baseline models

Controllers	Image input	Memory	Reflection
MPC	No	No	Closed-loop
LLM to Action	No	Yes	Open-loop
LLM-MPC	No	Yes	Closed-loop
VLM-MPC	Yes	Yes	Closed-loop

3.1.2 Experiment Results

Based on the experiment design, this section comprehensively evaluates the proposed VLM-MPC, focusing on safety, smoothness, and completion rate across various driving scenarios.

***3.1.2.1** Safety*

The safety evaluation results using PET as metrics are shown in TABLE 4. Both of our proposed controllers, LLM-MPC and VLM-MPC, demonstrate superior performance compared to the baseline LLM-action controller. Notably, LLM-MPC, without any task-specific fine-tuning, already outperforms the fine-tuned LLM-action baseline in terms of safety. This highlights that the benefit comes directly from integrating MPC into language model-based control rather than from additional training.

Furthermore, the VLM-MPC consistently achieves the best overall performance across all scenarios. It not only maintains PET values above the critical safety threshold of 1 second in every case but

also adapts more effectively under challenging conditions such as rain and nighttime. For instance, in the absence of rain and at non-intersections, VLM-MPC achieves PET values of 1.31 and 1.97, respectively, outperforming the real-world trajectory (1.65 and 1.33), the baseline MPC controller (1.73 and 1.74), and the baseline LLM-action controller (0.37 and 0.05). In more difficult scenarios such as rainy intersections, VLM-MPC continues to perform robustly with PET values of 1.36 and 1.92, again surpassing the other baselines.

The VLM-MPC's consistent ability to maintain PET above safety thresholds is not only due to the collision-avoidance logic embedded in the MPC's physical model, but also to the adaptive tuning of key parameters. In rainy or nighttime scenarios (Scenarios 5–8), the VLM tends to reduce the desired speed (vd) and increase the headway weight (Q_{h}), effectively lowering acceleration aggressiveness and extending braking distance. These adjustments enhance safety in low-visibility and high-risk environments, further establishing VLM-MPC as the strongest configuration among the tested methods.

TABLE 4 Minimum PET (s) for the grouped situation of proposed VLM-MPC and baseline models.

Scenario Number		1	2	3	4	5	6	7	8
Scenario property	Rain	0	0	0	0	1	1	1	1
	Night	0	0	1	1	0	0	1	1
	Intersection	0	1	0	1	0	1	0	1
Minimum PET (s) of different controller	Real-world	1.65	1.53	1.33	4.61	3.15	2.6	2.85	3.56
	MPC	1.12	1.1	1.33	1.26	1.91	1.62	1.65	1.2
	LLM to Action	0.45	1.59	1.26	1.36	1.36	2.65	0.75	1.84
	LLM-MPC	1.11	1.21	1.34	1.07	2.13	1.19	3.13	1.10
	VLM-MPC	1.31	1.18	1.97	1.03	2.68	1.36	3.53	1.92

Figure 6 demonstrates the application of the proposed VLM-MPC in a rainy intersection scenario in Boston. The Figure highlights the dynamic and adaptive nature of the VLM-MPC as it navigates the vehicle through the environment.

At $t = 0s$, the VLM agent receives the initial scene input, including basic instruction, reference memory, vehicle front camera image, environment description, and ego/preceding vehicle state. Based on this input, the VLM agent generates the initial key driving parameters [10,1.0,2.0,3.5,6.5,2.8]. These parameters are optimized to maintain safety and efficiency, considering the foggy weather and the presence of parked vehicles.

At $t = 5s$, the updated scenario shows the vehicle approaching an intersection with the preceding vehicle no longer detected. The ego vehicle's position and speed have changed, indicating it is getting closer to the intersection. The VLM agent recognizes this proximity to the intersection and updates the key driving parameters to [12,1,1.0,0.5,2.0]. The critical adjustment here is the reduction in the desired speed (from 6.5 m/s to 2.0 m/s). This change is crucial as it ensures the vehicle gradually slows down while approaching the intersection, enhancing safety by allowing it to stop smoothly if necessary.

By $t = 10s$, the vehicle is even closer to the intersection, and the VLM agent continuously updates the parameters to ensure the vehicle decelerates appropriately. The model's ability to adjust the desired speed based on real-time feedback demonstrates its effectiveness in handling complex driving scenarios. This adaptive behavior is essential for navigating intersections safely, especially under challenging weather conditions like rain. Due to space limitations, the vehicle's procedures at t=10 and t=15 are not shown in Figure 3.

The example case in Figure 6 illustrates the VLM-MPC system's capability to dynamically adjust key driving parameters in response to changing environmental and situational contexts, ensuring safe and efficient vehicle control. The system's proactive reduction in desired speed as the vehicle approaches the

intersection at $t = 5s$ exemplifies its intelligent and adaptive control strategy. If the vehicle keeps applying the driving parameters given by GPT at $t = 5s$, the large desired speed, it will not stop safely and smoothly before the stop line.

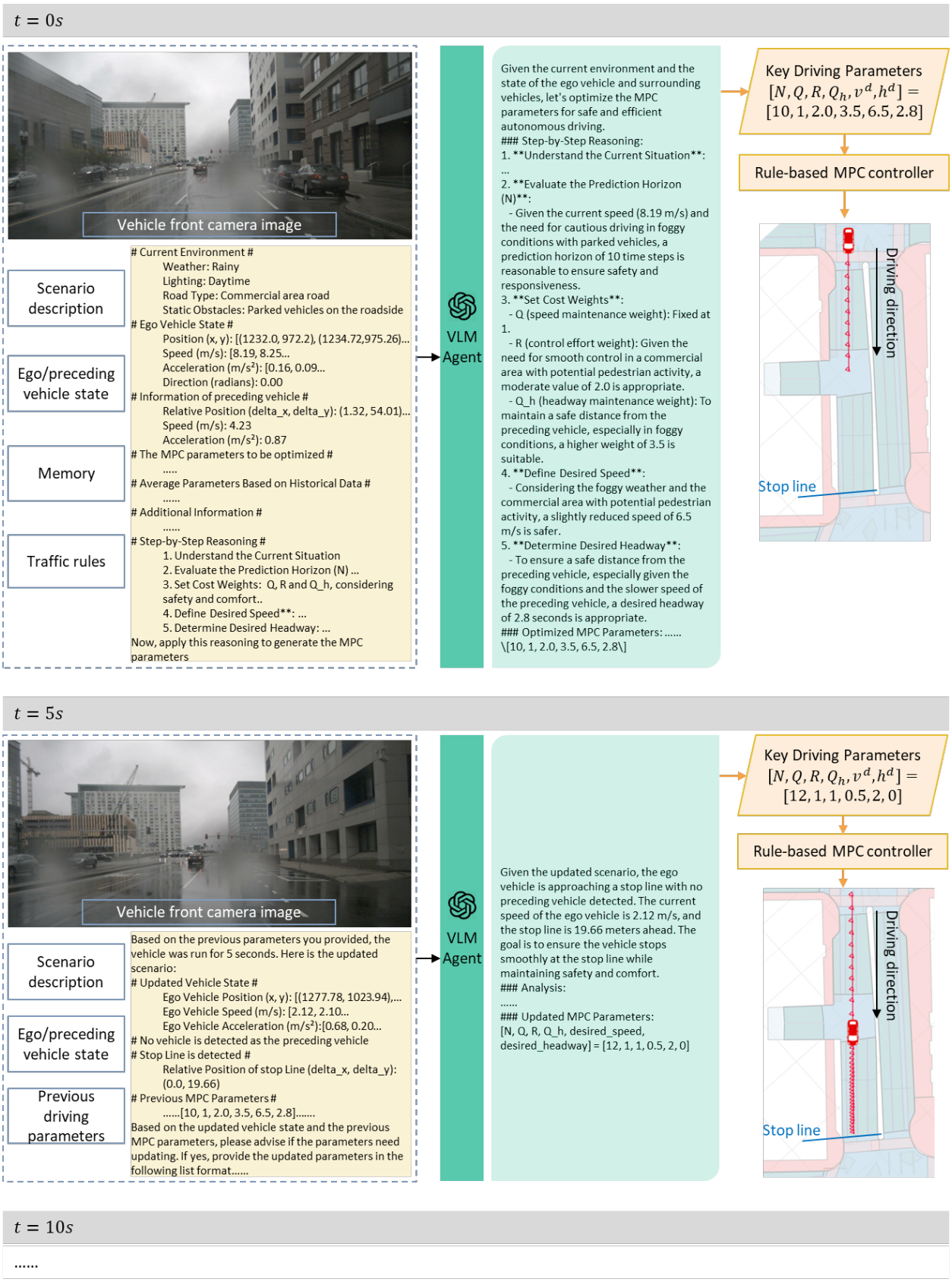
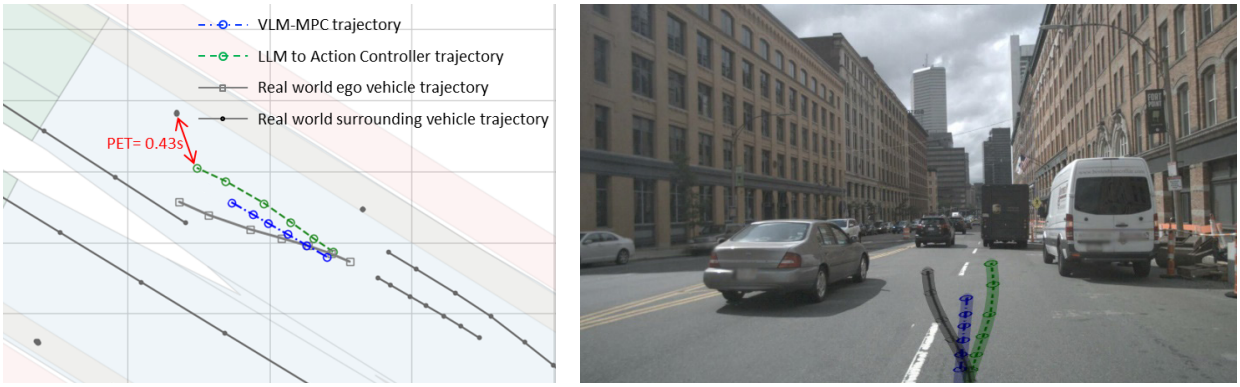
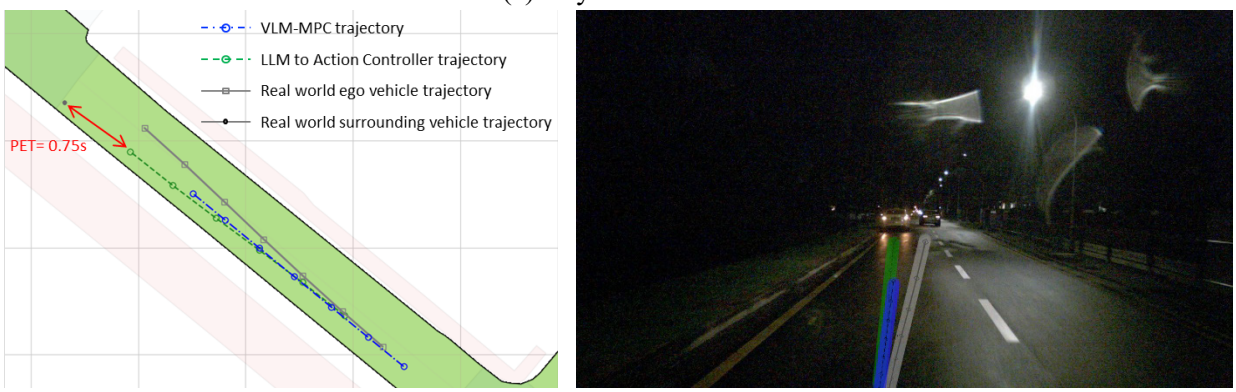


Figure 6 Illustration of the proposed VLM-MPC at one rainy intersection scenario in Boston.

Figure 7 illustrates two unsafe scenarios generated by the Baseline LLM to Action Controller. The trajectory shows snapshots from the final 3 seconds before a collision. In Figure 7 (a), during the daytime, the ego vehicle is approaching a situation where two vehicles are moving in the left lane ahead, while on the right, there are two parked vehicles and a work zone. In this case, the Baseline LLM to Action Controller plans a trajectory that advances forward, which is clearly unreasonable as it risks a collision with the parked cars. In contrast, the proposed VLM-MPC slows down to avoid potential collision. In Figure 7 (b), during nighttime, a similar issue arises: the LLM to Action Controller fails to account for a vehicle stopped at an intersection and instead accelerates towards it, resulting in a PET of 0.75 seconds, which is below the safety threshold. The proposed VLM-MPC, on the other hand, appropriately reduces speed to avoid collision.



(a) Daytime scene



(b) Nighttime scene

Figure 7 Comparison of the vehicle trajectory using proposed VLM-MPC and baseline LLM to Action controller

3.1.2.2 Smoothness

TABLE 5 illustrates the driving comfort of the proposed VLM-MPC system compared to baseline models by showing the RMS^a across various scenarios. The proposed VLM-MPC system consistently achieves the average lowest RMS^a values across most scenarios compared to the baseline models. This indicates that the VLM-MPC system provides smoother and more comfortable driving experiences. The reduction in RMS^a suggests that the VLM-MPC effectively minimizes abrupt changes in acceleration, leading to a more stable and pleasant ride for passengers.

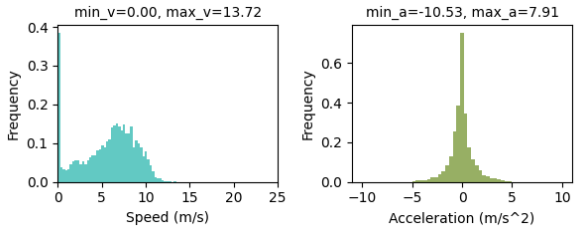
The Baseline LLM to Action Controller exhibits significantly highest RMS^a comparing to real-world trajectory and other models, due to the generated trajectories not aligning with vehicle dynamics. Figure 8 shows the distribution of speed and acceleration across different methods. It is evident that all the

proposed VLM-MPC, the baseline MPC, and the baseline LLM-MPC, maintain vehicle acceleration within the range of -3.5 to 3 m/s², which is achieved through lower-level MPC optimization, and their speed and acceleration distributions closely match the real-world trajectory. The real-world trajectory includes an extreme deceleration of -10.53 m/s² due to vehicle positioning errors. The Baseline LLM to Action Controller generates a trajectory with an extreme deceleration of -23.2 m/s². This highlights the inability of the LLM to directly plan a trajectory that adheres to vehicle kinematics and dynamics, making it difficult for the vehicle to execute the planned actions effectively.

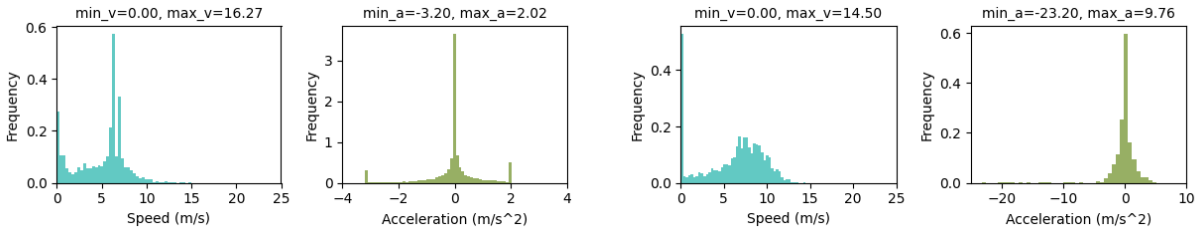
The improvement in smoothness (lower RMSa values) achieved by VLM-MPC is also linked to the dynamic selection control effort weight R, which affects how abruptly the MPC changes acceleration. In Scenarios 1 and 3, where traffic is less complex or visibility is better, VLM often outputs a slightly lower R, allowing more flexible control without compromising comfort. On the other hand, in congested or uncertain environments like Scenario 4 and 8, a higher R and Q^h combination promotes conservative behavior, avoiding jerky accelerations or unnecessary speed changes. This adaptive balancing between response and restraint directly explains the consistent RMS^a improvement observed in TABLE 5. Hence, VLM-MPC's smooth trajectory results stem not only from the MPC's internal optimization, but also from the well-calibrated parameter inputs based on real-time VLM reasoning.

TABLE 5 Smoothness RMS^a for the grouped situation of proposed VLM-MPC and baseline models.

Scenario Number		1	2	3	4	5	6	7	8
Scenarios	Rain	0	0	0	0	1	1	1	1
	Night	0	0	1	1	0	0	1	1
	Intersection	0	1	0	1	0	1	0	1
Smoothness RMS^a of different controller	Real-world	0.61	0.54	0.63	0.56	0.56	0.51	0.68	0.58
	MPC	0.41	0.44	0.47	0.51	0.4	0.42	0.35	0.46
	LLM to Action	0.43	0.93	0.94	1.41	2.14	2.18	3.13	1.66
	LLM-MPC	0.35	0.4	0.48	0.5	0.38	0.4	**0.33**	0.48
	VLM-MPC	**0.33**	**0.4**	**0.42**	**0.43**	**0.38**	**0.39**	0.34	**0.4**

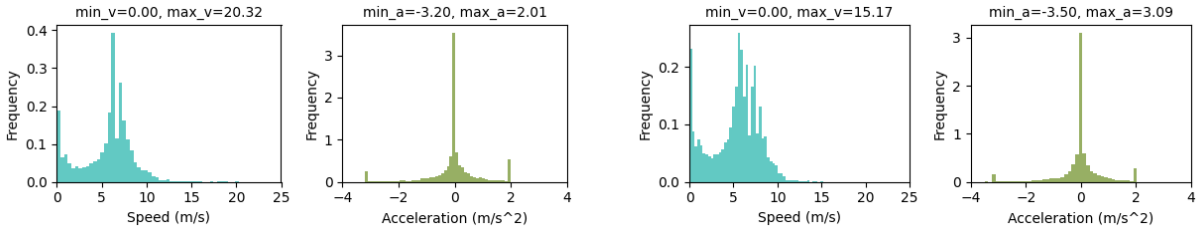


(a) Real-world



(b) MPC

(c) LLM to Action



(d) LLM-MPC (e) VLM-MPC

Figure 8 Distribution of speed and acceleration of proposed VLM-MPC and baseline models.

3.1.2.3 Completion Rate

VLM typically suffers from arbitrary predictions, which can produce invalid outputs (e.g., hallucinations or invalid formats) that are detrimental to driving systems. TABLE 6 listed the completion rate of proposed VLM-MPC using various LM Models and baseline models. The completion rate for Llava 1.6 across all scenarios was 99.7%, while the GPT series models (GPT-3.5-turbo, GPT-4-o, GPT-4-mini) achieved a 100% completion rate. This difference can be attributed to the significantly larger number of parameters in the GPT series models compared to Llava 1.6. Llava 1.6 uses a 7B parameter model, whereas GPT-3.5-turbo has 175B parameters. The GPT-4-o model is estimated to have between 100B and 1T parameters, and GPT-4-mini is estimated to have between 10B and 30B parameters. The larger parameter size in the GPT models enables them to handle a wider range of scenarios and produce more reliable outputs, reducing the likelihood of errors such as hallucinations or invalid format outputs. This robustness in handling diverse tasks contributes to the 100% completion rate observed in the GPT series models.

TABLE 6 Completion rate of different LM models

Models	LM Model	LM Model Parameter Size	Completion rate (%)
LLM-MPC	Llama-3.1	7B	99.7
	GPT-3.5-turbo	175B	100
LLM to Action	GPT-3.5-turbo	175B	100
VLM-MPC	Llava-1.6	7B	99.7
	GPT-4-o	100B~1T	100
	GPT-4-mini	10B~30B	100

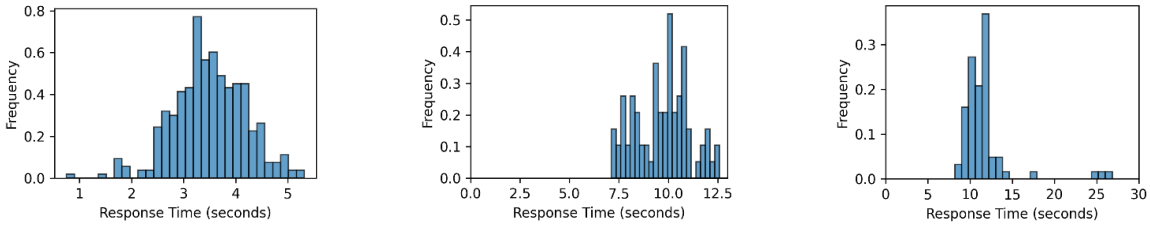
3.1.2.4 Response Time

To ensure consistent and interpretable response time measurements, we standardized the decoding configuration across all experiments. Specifically, we fixed the temperature to 0.0 and limited the maximum number of generated tokens to 100 for all models. These settings eliminate variability in latency caused by sampling behavior and allow us to attribute differences in response time (e.g., between daytime and nighttime scenes) primarily to visual input complexity. We observed that higher temperature values or a larger ‘max_new_tokens’ setting significantly increased inference time, especially under challenging conditions such as low-light scenes. The selected parameters represent the fastest stable configuration currently achievable given the capabilities of available VLM/LLMs.

Figure 9 illustrates the response speeds of the proposed VLM-MPC with different models. The Llava model has an average response time of 3.42 seconds, meeting the system's requirement of a 0.2Hz response speed. In contrast, the GPT-4-o and GPT-4-mini models have average response times of around 10 seconds, primarily because their parameter sizes are several orders of magnitude larger than Llava model. The Llava/Llama series models met the real-time response requirement and were therefore executed online during simulation. In contrast, the GPT-based models, which exceeded the response time threshold, were used in offline mode for comparison purposes.

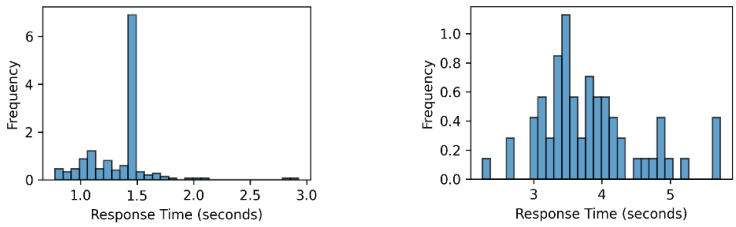
Figure 10 shows the response speeds of the baseline LLM-MPC with different models. Llama 3.1 and GPT-3.5-turbo models were tested. These models do not require image input, so the average response time is shorter than that of vision input models. The Llama 3.1 model has an average response time of 1.5 seconds, which is significantly faster than GPT-3.5-turbo's average response time of 3.9 seconds. This difference is also due to the smaller parameter size of Llama 3.1. The smaller parameter size allows for quicker processing and response times, making Llama 3.1 more suitable for applications where rapid decision-making is crucial. Conversely, while GPT-3.5-turbo has a larger parameter size that can handle more complex scenarios, it requires more computational resources, resulting in longer response times.

Overall, the response speed of the two locally running models (Llava and Llama) is faster than that of the cloud-based GPT models. This is primarily due to the difference in model parameter sizes, and also because the GPT models are affected by network latency and account token limits. In the future, we plan to fine-tune smaller parameter models to prepare for their application in vehicles.



Llava-1.6 GPT-4-o GPT-4-mini

Figure 9 Distribution of response times of proposed VLM-MPC



Llama3.1 GPT-3-turbo

Figure 10 Distribution of response times of baseline LLM-MPC

3.1.3 Sensitivity analysis

To validate the effectiveness of the proposed model, this section examines the impact of its key components on performance. And the sensitivity analysis on response time.

3.1.3.1 Impact of Reference Memory

To examine the importance of each module, we conducted ablation studies by selectively removing components from the proposed VLM-MPC framework. Specifically, we tested a variant VLM-MPC without Memory, which excludes the reference memory input to the VLM. As shown in TABLE 7, this model achieves a completion rate of 99.3%, slightly lower than the full model. Two cases failed due to the generation of control parameters outside the reasonable range. In scenarios with rain or night, the desired speed and headway values selected from memory are systematically lower. VLM learns to retrieve and generalize from the memory to form adaptive parameter decisions.

The response time remains around 1.5 seconds. In each scene, safety was maintained, as shown in TABLE 8. However, the parameters generated by the VLM generally favored higher speeds and larger headways. While the planned trajectories led to improved smoothness under clear weather conditions, the smoothness deteriorated in rainy and nighttime scenarios.

TABLE 7 Completion rate when using Llava-1.6.

Ablation models	Completion rate (%)
VLM-MPC without Environment encoder	87.5
VLM-MPC without Memory	99.3
VLM-MPC	99.7

TABLE 8 Smoothness RMS^a for the grouped situation of proposed VLM-MPC and baseline models

Scenario Number		1	2	3	4	5	6	7	8
Scenarios	Rain	0	0	0	0	1	1	1	1
	Night	0	0	1	1	0	0	1	1
	Intersection	0	1	0	1	0	1	0	1
Minimum PET (s) of different controller	VLM-MPC without Memory	2.53	1.36	1.86	1.39	2.14	2.32	14.95	1.20
	VLM-MPC without Environment encoder	1.66	1.13	1.58	3.70	2.28	1.91	1.65	1.20
	VLM-MPC	1.31	1.18	1.97	1.03	2.68	1.36	3.53	1.92
Smoothness RMS^a of different controller	VLM-MPC without Memory	0.35	0.40	0.41	0.46	0.35	0.30	0.74	0.95
	VLM-MPC without Environment encoder	0.37	0.40	0.47	0.35	0.36	0.36	0.77	0.91
	VLM-MPC	0.33	0.4	0.42	0.43	0.38	0.39	0.33	0.4

3.1.3.2 Impact of Environment Description

Another ablation model, VLM-MPC without Environment Encoder, omits the environmental description (e.g., weather, lighting, road condition) from the prompt fed into the VLM, the completion rate drops significantly to 87.5%, much lower than the proposed VLM-MPC method, shown in TABLE 7. The failure cases include: The VLM explicitly stating it cannot understand the scene; Generating no output parameters; Producing a matrix instead of a single value for one parameter; Providing only a range instead of a numerical value for a parameter. These results indicate that the environment encoder provides essential contextual information that helps the VLM stably produce valid and well-structured outputs.

In addition, omitting environmental input significantly slowed down response time, with some scenes requiring more than 200 seconds to generate results. This further highlights the importance of environmental cues for reducing inference uncertainty and improving response efficiency.

Despite the drop in completion rate, the model still maintained safe behavior in scenarios where valid parameters were eventually generated. Compared to end-to-end models and real-world trajectories, the VLM-MPC (even with ablations) still offered better safety margins in those successful trials.

3.1.3.3 Interpretability analysis

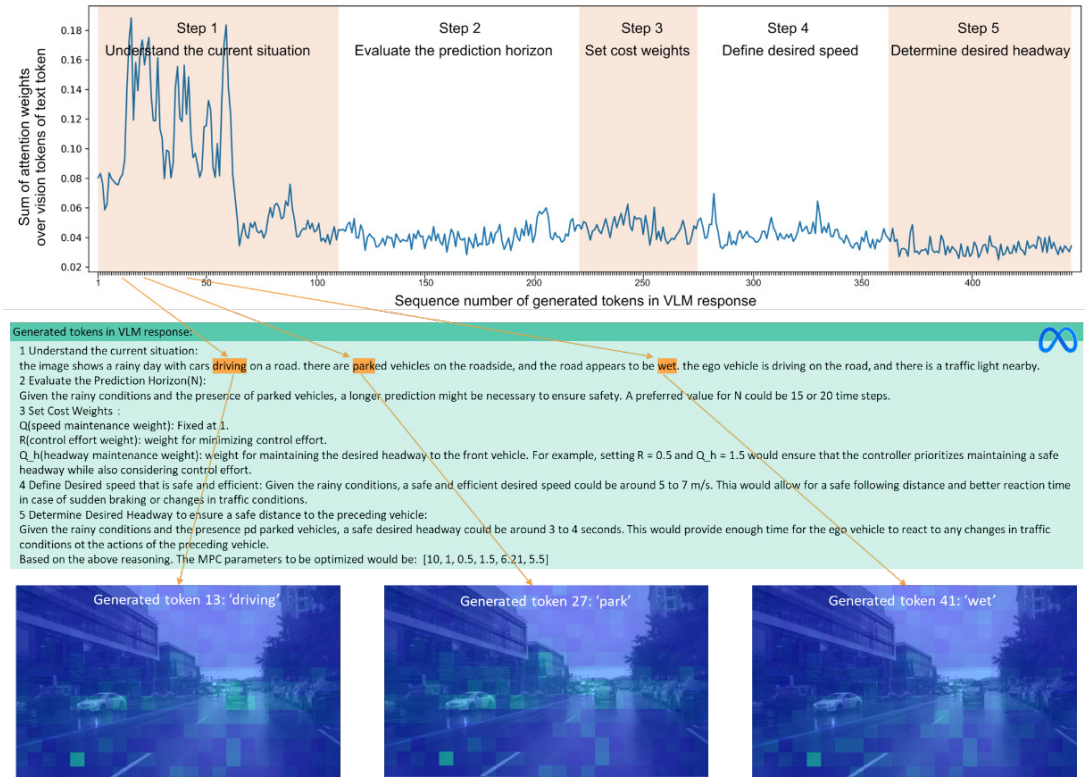
To facilitate a more intuitive and comprehensible interpretation of the VLM-MPC, aiding in the analysis of the causal relationships between the environment and VLM outputs. We demonstrate this with two selected scenarios.

First, we extracted the model's attention allocation to the image input while generating each token. A higher sum of attention weights indicates a greater focus on the image content when generating that particular token (zjysteven, 2024). Figure 11 presents the sum of attention the model assigns to the image input during the generation of each token, with the horizontal axis representing the tokens in sequence. The results reveal a consistent pattern across the two scenarios: during the first 50-60 tokens, approximately 14% of the attention is directed towards the vision tokens, significantly higher than in subsequent tokens. This is aligned with our implementation of CoT, where the initial step is ‘Understand the current environment’.

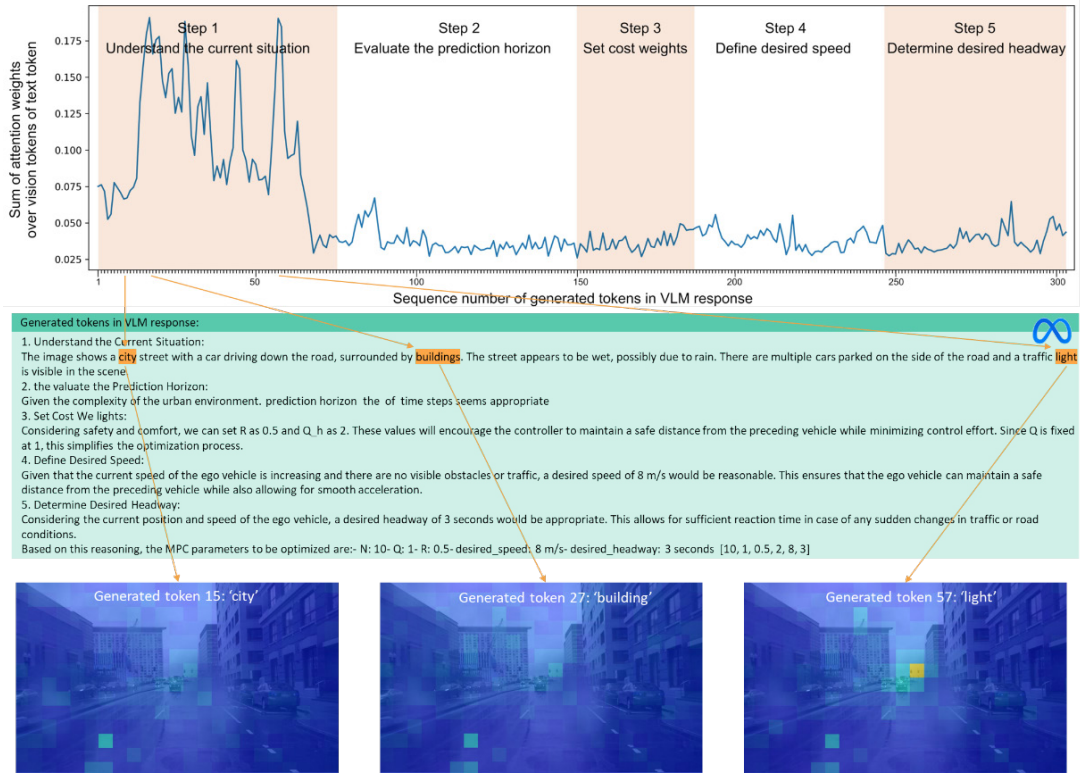
Consequently, the model's initial 50-60 tokens are dedicated to describing the environment, naturally leading to a stronger correlation with the image input. The subsequent tokens correspond to steps two through five of the CoT: 'Evaluate the prediction horizon', 'Set cost weights', 'Define desired speed', and 'Determine desired headway'. These steps are more inference-focused and rely less on the image input, drawing more from textual information.

Subsequently, we further integrated the vision encoder to visualize the attention map over the input image. The vision encoder in the model processes the image input and generates attention weights across different layers. By averaging the attention weights across all layers, we obtained a comprehensive attention matrix. For each key phrase generated, we combined the model's attention weights with the vision encoder's attention matrix to compute the model's focus on different regions of the image during the generation of that phrase. The computed attention values were then mapped onto the input image (vehicle front camera image), highlighting the specific regions the model attended to in the image.

Figure 11 illustrates the results for two representative scenarios, where we selected the first few tokens generated by the model and the corresponding short phrases, along with the attention allocated to the image during their generation. In the first scenario, when the model outputs "… with vehicles driving on a road…", the attention is concentrated on the road ahead. When the model generates "…there are parked vehicles…", more attention is directed towards the parked vehicles on the side of the road. When it outputs "…The road appears to be wet…", the attention becomes more dispersed, with less focus on the vehicles. In the second scenario, when the model generates "…a traffic light is visible…", the traffic light in the image clearly receives the highest attention. This demonstrates that the model indeed focuses on relevant parts of the image when generating these descriptions, highlighting the importance of the visual input.



(a) Scene 1



(b) Scene 2

Figure 11 Sum of attention the model pays to the image input when generating each token.

3.2 Carla-based Simulation: Adverse Scenarios Close-loop control

3.2.1 Experiment Setup

Although the nuScenes dataset offers the advantage of real-world driving data, it has several limitations: (1) each scene is relatively short (approximately 20 seconds), and (2) the dataset lacks extreme conditions such as heavy fog, heavy rain, and high pedestrian density. To address these limitations, we supplement our experiments with controlled simulations in the CARLA simulator (Dosovitskiy et al., 2017). CARLA supports long-horizon autonomous driving in dynamic and interactive environments, enabling us to simulate rare but safety-critical conditions such as foggy nights, jaywalking pedestrians, and work zones. The detailed simulation setting is shown in TABLE 9

In the CARLA-based simulation setup, a set of controlled driving scenarios across seven distinct weather conditions are designed, illustrated in Figure 12. The simulations are conducted using Town02 and Town05 maps to emulate underrepresented or unrealistic weather and pedestrian behaviors such as red-light violations by pedestrians. Pedestrian and vehicle flows are randomly generated to ensure scene diversity. These scenarios are challenging long-tail conditions that can be predefined in CARLA, we recognize that they are still limited compared to the long-tail situations observed in real-world driving. In future work, we plan to explore systematic approaches for generating complex and rare scenarios, such as adversarial scenario generation, to further test the robustness of the proposed framework. This setup also ensures the use of non-public, synthetic data that is highly unlikely to exist in any VLM's pretraining dataset.

Each simulation begins with the ego vehicle at a fixed starting point and follows a predefined looped route. For each weather configuration, three repeated runs are conducted, each lasting 240 seconds. This long-horizon setting allows us to evaluate the robustness and adaptability of the proposed VLM-MPC framework under diverse and adverse conditions.

All non-ego vehicles are controlled by Carla's rule-based autonomous agents, which respond to traffic signals and the ego vehicle's position and movement. This enables a degree of realistic interaction during simulation.



Figure 12 Front camera view of ego vehicle of 7 scenarios in CARLA Simulation

TABLE 9 Weather Parameter Settings in CARLA Simulation

Key settings	Clear daytime	Light rain	Heavy rain	Night	Rainy night	Light fog	Heavy fog
Cloudiness (%)	0	60	90	0	100	40	90
Precipitation (%)	0	30	100	0	100	0	0

Precipitation deposits (%)	0	40	80	0	100	0	0
Wind Intensity (%)	0	20	50	0	60	0	0
Fog Density (%)	0	0	0	0	0	60	90
Fog Distance (m)	0	0	0	0	0	100	20

3.2.2 Experiment results

3.2.2.1 Safety

Results concluded in TABLE 10 show that scenarios such as heavy rain and heavy fog result in higher PET values. The VLM becomes more cautious in adverse conditions, generating lower desired speeds and longer headways. For example, as shown in Figure 13(b), upon detecting heavy rain and pedestrians crossing the road, the VLM immediately triggers deceleration. Similarly, in Figure 13(c), even in the absence of visible vehicles, the VLM—guided by the environment encoder recognizing heavy fog—slows down and maintains a large headway. In contrast, the system behaves similarly under clear, light rain, and light fog conditions. This is due to the fact that CLIP occasionally fails to recognize these mild weather changes in CARLA, where the rendering realism of light rain/fog is limited. However, since these mild conditions do not significantly impact safety, the model does not deliberately slow down, which is considered acceptable.

3.2.2.2 Smoothness

Given the long simulation time, the ego vehicle is required to perform frequent start-stop maneuvers at intersections. In scenarios such as heavy rain, heavy fog, and rainy night, due to the lower desired speeds and cautious accelerations, the RMS acceleration is noticeably lower, resulting in smoother trajectories.

3.2.2.3 Completion Rate

Note that the completion rate here does not refer to route completion, but rather the success rate of VLM responses when queried. Even when a VLM call fails to produce valid parameters, the VLM-MPC framework can fall back to previous outputs or reference memory, ensuring continued operation. Using LLaVA 1.6, we observed slightly lower completion rates compared to nuScenes, mainly due to Carla scenes being visually simpler and more open, which sometimes leads the model to conclude that parameters cannot be reliably determined.

3.2.2.4 Response Time

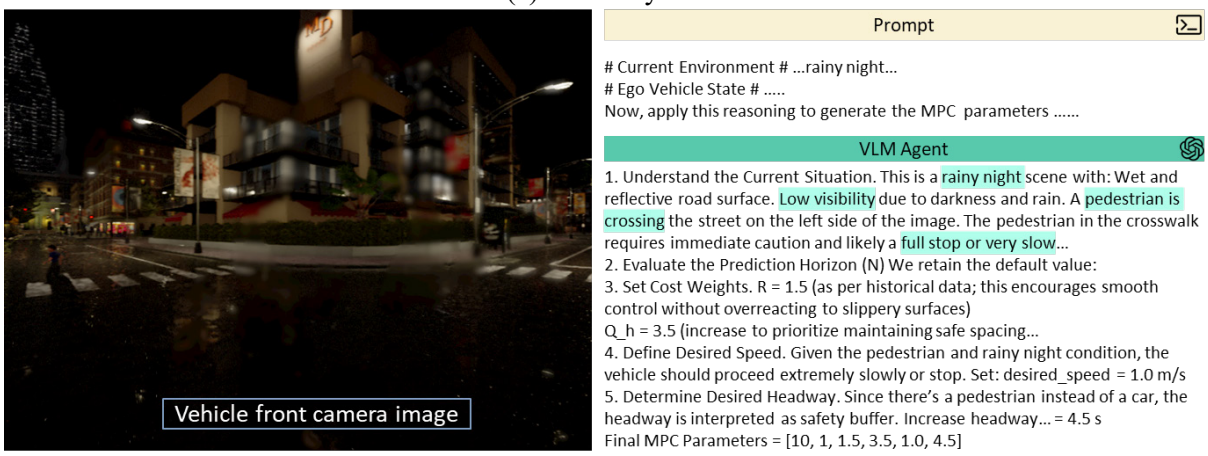
Across all weather conditions, the average VLM response time remains around 2 seconds. This indicates that each VLM parameter update introduces approximately a 2-second delay before passing the result to the MPC controller.

TABLE 10 experiment results of Carla of VLM-MPC

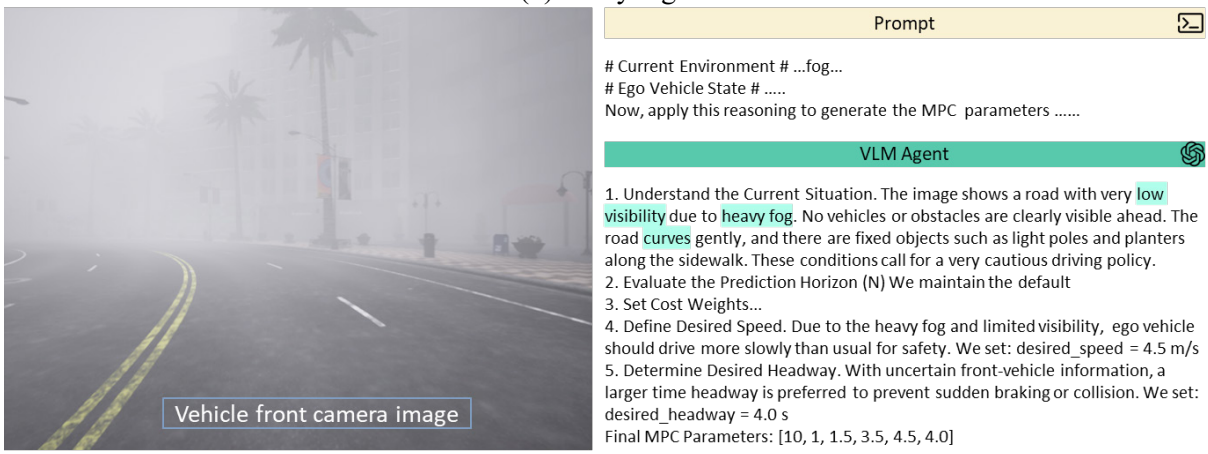
	Clear daytime	Light rain	Heavy rain	Night	Rainy night	Light fog	Heavy fog
Minimum PET (s) of different controller	2.57	1.74	3.02	2.57	2.32	2.37	2.82
Smoothness RMS^a of different controller	2.64	2.70	2.85	3.18	2.67	3.03	2.82
Completion rate	93.75%	92.18%	89.15%	90.8%	94.62%	91.41%	91.67%
Average Response Time (Variance)	2.09 (0.0039)	2.073 (0.0041)	2.11 (0.0043)	2.066 (0.0017)	2.092 (0.0021)	2.102 (0.0025)	2.068 (0.0018)



(a) Clear daytime



(b) Rainy night



(c) Heavy fog

Figure 13 VLM understanding in various environments

4 CONCLUSION AND FUTURE RESEARCH

This paper introduced a closed-loop autonomous driving controller, VLM-MPC, that leverages VLMs for upper-level decisions and MPC for low-level vehicle control. This design enables the upper-level model to provide adaptive decision-making based on scenario changes, while the lower level controls the vehicle in real-time based on the current state of the ego vehicle and surrounding vehicles.

In experiments based on the nuScenes dataset, we compared this structure with the method of using an LLM to control the vehicle independently. The results validate that a model-based approach like MPC significantly enhances the performance of LLM-based controllers. The integration of MPC allows the system to make more accurate and reliable decisions, especially in dynamic and complex driving environments, demonstrating the advantages of combining model-based methods with LLMs for autonomous driving. Experiment results demonstrate that the VLM-MPC consistently maintains PET above safe thresholds, in contrast to some scenarios where the VLM-based control posed collision risks. Additionally, the VLM-MPC enhances smoothness compared to the real-world trajectories and VLM-based control. By comparing behaviors under different environmental settings, we highlight the VLM-MPC's capability to understand the environment and make reasoned inferences. VLM-MPC with Llava could reach the response time requirements of the proposed method. Moreover, we validate the significant contributions of two key components, the reference memory and the environment encoder, to the stability of responses through ablation tests. We further conduct extended simulations in the CARLA environment to evaluate the model's robustness under long-horizon and adverse conditions such as fog and rainy night.

Future research will focus on two main directions. The first direction is to refine the proposed approach with more ablation studies to validate the effectiveness of each component and further optimize the system. This includes investigating the relationship between the VLM's image input understanding capabilities and the subsequent driving behavior choices, assessing the impact of memory on the stability of the VLM-generated results, and evaluating how the frequency of the asynchronous operation between the upper and lower levels affects overall vehicle performance.

Second, we will broaden the scope of validation to improve generalizability. In particular, we plan to extend our experiments to VLM-action baselines by fine-tuning VLMs to directly generate actions, with a focus on mitigating vision hallucination. We also aim to evaluate the proposed framework in more complex and longer-horizon scenarios, both in simulation (e.g., highway driving, dense urban flows, adverse weather) and real-world vehicle experiments. Collecting real-world data under varied conditions will allow us to better analyze the impact of driving environments on decision-making, identify out-of-distribution (ODD) scenarios, and conduct more rigorous closed-loop testing that captures cumulative uncertainties from perception, reasoning, and low-level control.

Third, we will investigate hallucinations in LLM-based driving systems at a finer granularity. Specifically, we aim to disentangle how hallucinations arise at different stages—perception, reasoning, and decision-making—and how they affect overall control performance. Understanding these stage-specific failure modes is crucial for building safer and more reliable LLM-driven controllers. Inspired by emerging Vision-Language-Action (VLA) models, we also plan to explore whether LLMs with general knowledge can serve as the backbone of future driving agents, potentially enabling more explainable autonomous driving.

5 ACKNOWLEDGMENT

This work was supported by the National Science Foundation Cyber-Physical Systems (CPS) program. Award Number: 2343167.

6 REFERENCES

Bai, Y., Wu, D., Liu, Y., Jia, F., Mao, W., Zhang, Z., Zhao, Y., Shen, J., Wei, X., Wang, T., Zhang, X., 2024. Is a 3D-Tokenized LLM the Key to Reliable Autonomous Driving?

Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D.M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., Amodei, D., 2020. Language Models are Few-Shot Learners. https://doi.org/10.48550/arXiv.2005.14165

Caesar, H., Bankiti, V., Lang, A.H., Vora, S., Liong, V.E., Xu, Q., Krishnan, A., Pan, Y., Baldan, G.,

Beijbom, O., 2020. nuScenes: A Multimodal Dataset for Autonomous Driving. Presented at the Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 11621–11631.

Campolo, C., Molinaro, A., Araniti, G., Berthet, A.O., 2017. Better Platooning Control Toward Autonomous Driving : An LTE Device-to-Device Communications Strategy That Meets Ultralow Latency Requirements. IEEE Vehicular Technology Magazine 12, 30–38. https://doi.org/10.1109/MVT.2016.2632418

Chen, L., Sinavski, O., Hünermann, J., Karnsund, A., Willmott, A.J., Birch, D., Maund, D., Shotton, J., 2023. Driving with LLMs: Fusing Object-Level Vector Modality for Explainable Autonomous Driving.

Chen, Y., Ding, Z., Wang, Z., Wang, Y., Zhang, L., Liu, S., 2024. Asynchronous Large Language Model Enhanced Planner for Autonomous Driving.

Cui, C., Ma, Y., Cao, X., Ye, W., Wang, Z., 2024a. Receive, Reason, and React: Drive as You Say, With Large Language Models in Autonomous Vehicles. IEEE Intelligent Transportation Systems Magazine 16, 81–94. https://doi.org/10.1109/MITS.2024.3381793

Cui, C., Yang, Z., Zhou, Y., Ma, Y., Lu, J., Li, L., Chen, Y., Panchal, J., Wang, Z., 2024b. Personalized Autonomous Driving with Large Language Models: Field Experiments. https://doi.org/10.48550/arXiv.2312.09397

Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., Koltun, V., 2017. CARLA: An Open Urban Driving Simulator, in: Proceedings of the 1st Annual Conference on Robot Learning. Presented at the Conference on Robot Learning, PMLR, pp. 1–16.

Fang J., Li L., Zhou J., Xiao J., Yu H., Lv C., Xue J., Chua T.-S., 2024. Abductive Ego-View Accident Video Understanding for Safe Driving Perception. Presented at the Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 22030–22040.

Farquhar, S., Kossen, J., Kuhn, L., Gal, Y., 2024. Detecting hallucinations in large language models using semantic entropy. Nature 630, 625–630. https://doi.org/10.1038/s41586-024-07421-0

Filos, A., Tigkas, P., Mcallister, R., Rhinehart, N., Levine, S., Gal, Y., 2020. Can Autonomous Vehicles Identify, Recover From, and Adapt to Distribution Shifts?, in: Proceedings of the 37th International Conference on Machine Learning. Presented at the International Conference on Machine Learning, PMLR, pp. 3145–3153.

Gao, Z., Wu, Z., Hao, W., Long, Keke, Byon, Y.-J., Long, Kejun, 2022. Optimal Trajectory Planning of Connected and Automated Vehicles at On-Ramp Merging Area. IEEE Transactions on Intelligent Transportation Systems 23, 12675–12687. https://doi.org/10.1109/TITS.2021.3116666

Guo, Z., Lykov, A., Yagudin, Z., Konenkov, M., Tsetserukou, D., 2024. Co-driver: VLM-based Autonomous Driving Assistant with Human-like Behavior and Understanding for Complex Road Scenes.

Huang, Y., Sansom, J., Ma, Z., Gervits, F., Chai, J., 2024. DriVLMe: Enhancing LLM-based Autonomous Driving Agents with Embodied and Social Experiences.

Hwang, J.-J., Xu, R., Lin, H., Hung, W.-C., Ji, J., Choi, K., Huang, D., He, T., Covington, P., Sapp, B., Guo, J., Anguelov, D., Tan, M., 2024. EMMA: End-to-End Multimodal Model for Autonomous Driving [WWW Document]. arXiv.org. URL https://arxiv.org/abs/2410.23262v1 (accessed 5.22.25).

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y., Chen, D., Dai, W., Chan, H.S., Madotto, A., Fung, P., 2023. Survey of Hallucination in Natural Language Generation. ACM Comput. Surv. 55, 1–38. https://doi.org/10.1145/3571730

Jia, F., Mao, W., Liu, Y., Zhao, Y., Wen, Y., Zhang, C., Zhang, X., Wang, T., 2023. ADriver-I: A General World Model for Autonomous Driving. https://doi.org/10.48550/arXiv.2311.13549

Jin, Y., Shen, X., Peng, H., Liu, X., Qin, J., Li, J., Xie, J., Gao, P., Zhou, G., Gong, J., 2023. SurrealDriver: Designing Generative Driver Agent Simulation Framework in Urban Contexts based on Large Language Model.

Johnson, J., Douze, M., Jégou, H., 2021. Billion-Scale Similarity Search with GPUs. IEEE Transactions on

Big Data 7, 535–547. https://doi.org/10.1109/TBDATA.2019.2921572
Li, B., Wang, Y., Mao, J., Ivanovic, B., Veer, S., Leung, K., Pavone, M., 2024. Driving Everywhere with Large Language Model Policy Adaptation [WWW Document]. arXiv.org. URL https://arxiv.org/abs/2402.05932v2 (accessed 5.23.25).
Liu, H., Li, C., Li, Y., Li, B., Zhang, Y., Shen, S., Lee, Y.J., 2024. LLaVA-NeXT: Improved reasoning, OCR, and world knowledge.
Long, K., Liang, Z., Shi, H., Shi, L., Chen, S., Li, X., 2024. Traffic oscillation mitigation with physics-enhanced residual learning (PERL)-based predictive control. Communications in Transportation Research 4, 100154. https://doi.org/10.1016/j.commtr.2024.100154
Ma, Yingzi, Cao, Y., Sun, J., Pavone, M., Xiao, C., 2023. Dolphins: Multimodal Language Model for Driving. https://doi.org/10.48550/arXiv.2312.00438
Ma, Yunsheng, Cui, C., Cao, X., Ye, W., Liu, P., Lu, J., Abdelraouf, A., Gupta, R., Han, K., Bera, A., Rehg, J.M., Wang, Z., 2023. LaMPilot: An Open Benchmark Dataset for Autonomous Driving with Language Model Programs.
Mao, J., Qian, Y., Ye, J., Zhao, H., Wang, Y., 2023. GPT-Driver: Learning to Drive with GPT. https://doi.org/10.48550/arXiv.2310.01415
Mao, J., Ye, J., Qian, Y., Pavone, M., Wang, Y., 2024. A Language Agent for Autonomous Driving. https://doi.org/10.48550/arXiv.2311.10813
Peesapati, L.N., Hunter, M.P., Rodgers, M.O., 2018. Can post encroachment time substitute intersection characteristics in crash prediction models? Journal of Safety Research 66, 205–211. https://doi.org/10.1016/j.jsr.2018.05.002
Radford, A., Kim, J.W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., Sutskever, I., 2021. Learning Transferable Visual Models From Natural Language Supervision. https://doi.org/10.48550/arXiv.2103.00020
Reiter, R., Diehl, M., 2021. Parameterization Approach of the Frenet Transformation for Model Predictive Control of Autonomous Vehicles, in: 2021 European Control Conference (ECC). Presented at the 2021 European Control Conference (ECC), pp. 2414–2419. https://doi.org/10.23919/ECC54610.2021.9655053
Sha, H., Mu, Y., Jiang, Y., Chen, L., Xu, C., Luo, P., Li, S.E., Tomizuka, M., Zhan, W., Ding, M., 2023. LanguageMPC: Large Language Models as Decision Makers for Autonomous Driving. https://doi.org/10.48550/arXiv.2310.03026
Shang, M., Rosenblad, B., Stern, R., 2022. A Novel Asymmetric Car Following Model for Driver-Assist Enabled Vehicle Dynamics. IEEE Transactions on Intelligent Transportation Systems 23, 15696–15706. https://doi.org/10.1109/TITS.2022.3145292
Shao, H., Hu, Y., Wang, L., Waslander, S.L., Liu, Y., Li, H., 2023. LMDrive: Closed-Loop End-to-End Driving with Large Language Models. https://doi.org/10.48550/arXiv.2312.07488
Sharan, S.P., Pittaluga, F., G, V.K.B., Chandraker, M., 2023. LLM-Assist: Enhancing Closed-Loop Planning with Language-Based Reasoning. https://doi.org/10.48550/arXiv.2401.00125
Shi, H., Zhou, Y., Wu, K., Wang, X., Lin, Y., Ran, B., 2021. Connected automated vehicle cooperative control with a deep reinforcement learning approach in a mixed traffic environment. Transportation Research Part C: Emerging Technologies 133, 103421. https://doi.org/10.1016/j.trc.2021.103421
Tian, X., Gu, J., Li, B., Liu, Y., Wang, Y., Zhao, Z., Zhan, K., Jia, P., Lang, X., Zhao, H., 2024. DriveVLM: The Convergence of Autonomous Driving and Large Vision-Language Models.
Valmeekam, K., Marquez, M., Kambhampati, S., 2023. Can Large Language Models Really Improve by Self-critiquing Their Own Plans? https://doi.org/10.48550/arXiv.2310.08118
Wang, P., Zhu, M., Zheng, X., Lu, H., Zhong, H., Chen, X., Shen, S., Wang, X., Wang, Y., Wang, F.-Y., 2024. BEVGPT: Generative Pre-trained Foundation Model for Autonomous Driving Prediction, Decision-Making, and Planning. IEEE Transactions on Intelligent Vehicles 1–13. https://doi.org/10.1109/TIV.2024.3449278
Wang, S., Yu, Z., Jiang, X., Lan, S., Shi, M., Chang, N., Kautz, J., Li, Y., Alvarez, J.M., 2024. OmniDrive: A Holistic LLM-Agent Framework for Autonomous Driving with 3D Perception, Reasoning and

Planning.
Wang, Y., Xing, S., Can, C., Li, R., Hua, H., Tian, K., Mo, Z., Gao, X., Wu, K., Zhou, S., You, H., Peng, J., Zhang, J., Wang, Zehao, Song, R., Yan, M., Zimmer, W., Zhou, X., Li, P., Lu, Z., Chen, C.-J., Huang, Y., Rossi, R.A., Sun, L., Yu, H., Fan, Z., Yang, F.H., Kang, Y., Greer, R., Liu, C., Lee, E.H., Di, X., Ye, X., Ren, L., Knoll, A., Li, X., Ji, S., Tomizuka, M., Pavone, M., Yang, T., Du, J., Yang, M.-H., Wei, H., Wang, Ziran, Zhou, Y., Li, J., Tu, Z., 2025. Generative AI for Autonomous Driving: Frontiers and Opportunities [WWW Document]. arXiv.org. URL https://arxiv.org/abs/2505.08854v1 (accessed 5.22.25).
Wang, Z., Zhao, X., Xu, Z., Li, X., Qu, X., 2021. Modeling and field experiments on autonomous vehicle lane changing with surrounding human-driven vehicles. Computer-Aided Civil and Infrastructure Engineering 36, 877–889. https://doi.org/10.1111/mice.12540
Wen, L., Fu, D., Li, X., Cai, X., Ma, T., Cai, P., Dou, M., Shi, B., He, L., Qiao, Y., 2023. DiLu: A Knowledge-Driven Approach to Autonomous Driving with Large Language Models. Presented at the The Twelfth International Conference on Learning Representations.
Xu, Z., Zhang, Y., Xie, E., Zhao, Z., Guo, Y., Wong, K.-Y.K., Li, Z., Zhao, H., 2024. DriveGPT4: Interpretable End-to-end Autonomous Driving via Large Language Model.
Zheng, Yupeng, Xing, Z., Zhang, Q., Jin, B., Li, P., Zheng, Yuhang, Xia, Z., Zhan, K., Lang, X., Chen, Y., Zhao, D., 2024. PlanAgent: A Multi-modal Large Language Agent for Closed-loop Vehicle Motion Planning. https://doi.org/10.48550/arXiv.2406.01587
zjysteven, 2024. zjysteven/VLM-Visualizer: Visualizing the attention of vision-language models [WWW Document]. URL https://github.com/zjysteven/VLM-Visualizer (accessed 8.26.24).